Context for AI: CacheDeal Android App

Instructions for the AI: I am providing you with the complete context of an Android application I developed called "CacheDeal". Please use this information to help me write a highly engaging, professional section for my GitHub Profile README (the special repository that shows up on my GitHub profile). I want this project to be a major highlight of my profile.

---

1. High-Level Summary

- **App Name:** CacheDeal
- **Platform:** Native Android
- **Target Audience:** University students (specifically built for VIT Vellore).
- **Core Concept:** A hyper-local campus marketplace where students can buy and sell essentials (cycles, lab coats, notes, game accounts) to peers nearby.

2. The Problem It Solves

Currently, campus trading happens in chaotic, fragmented WhatsApp or Telegram groups.

- Listings get buried under hundreds of messages.
- Negotiations are disorganized.
- **The biggest issue:** Buyers frequently "ghost" or flake on meetups with zero consequences because everything is anonymous and untracked.

3. The CacheDeal Solution & Features

CacheDeal centralizes campus commerce and introduces strict accountability.

- **Structured Listings:** Sellers post items under specific fixed categories with a photo, title, and asking price.
- **Standardized Bidding:** Buyers browse the feed (with optional "Near Me / Same Hostel Block" filtering) and submit structured cash offers with optional notes.
- **Offer Review & Locking:** The seller sees a list of all incoming offers sorted by price. When the seller accepts an offer, the system executes an atomic transaction (Firestore Batch Write) to lock the deal, reject all other offers, and generate a WhatsApp deep-link so the two parties can arrange a meetup.
- **The Reputation System (Anti-Ghosting):**
    - Every user has a public "Green Dot" (successful deals) and "Red Dot" (flaked deals) score.
    - When a deal is locked, a 3-day countdown begins.
    - If both parties tap "Mark as Completed", both earn a Green Dot.
    - If the buyer ghosts, the seller can tap "Re-list". This automatically penalizes the buyer with a Red Dot.
    - Sellers can see a buyer's dot score *before* accepting their offer, meaning a slightly lower offer from a highly reliable buyer (many green dots) might win over a high offer from a flaky buyer (many red dots).

4. Technical Stack & Architecture

- **Language:** Kotlin
- **UI Framework:** Jetpack Compose (Modern, declarative UI with Material Design 3)
- **Architecture:** MVVM (Model-View-ViewModel) utilizing Kotlin Coroutines and Flows for asynchronous state management.

- **Backend (BaaS):** Firebase
  - **Firebase Authentication:** Handles secure user sign-ups (Email/Password) with custom domain constraints (e.g., restricted to `@vitstudent.ac.in`).
  - **Cloud Firestore:** A NoSQL real-time database managing user profiles, item listings, nested offers, and deal states.
  - **Firebase Storage:** Securely handles user-uploaded item photos.
- **Image Loading:** Coil (Async image loading library for Compose).

5. Key Engineering Highlights to Showcase

- **Concurrency Handling:** Used Firestore Batch Writes to prevent race conditions during the deal-locking phase (preventing two fast offers from being accepted simultaneously).
- **Reactive UI:** Leveraged Jetpack Compose's state-hoisting and animations to create a fluid, modern interface that reacts instantly to backend database changes without manual DOM manipulation.
- **Automated Trustless Accountability:** Engineered the Red-Dot penalty as an automated side-effect of the seller's "re-list" action, eliminating the need for a manual (and easily abused) user-reporting system.